PP1 - Vyhodnocování úplnosti testů vůči softwarovým požadavkům - formalizmus

April 2026

1 Primitive Domains

Domain	Symbol	Definition
Time	**T**	Totally ordered set; one flavour chosen per requirement.
Values	**V**	Booleans, integers, reals, enumerations, bit-vectors, structured records.

For each requirement exactly one interpretation of **T** is fixed:

Flavour	Carrier	Use
Logical	Countable ordered set, no metric	ordering only
Metric	$\mathbf{T} = \mathbb{N}$	discrete steps
Real	$\mathbf{T} = \mathbb{R}_{\geq 0}$	physical deadlines

2 The Dictionary

Definition 1 (Dictionary)**.** *A* dictionary *is a set*

$$D = \{\, (sem,\ ops,\ mods) \,\}$$

where each triple consists of a semantics sem, a set of operations ops (each with an associated effect predicate), and a modifier schema mods. D is open: new triples may be added without altering the core.

2.1 Built-in types

Type	Operations	Modifiers
`SIGNAL`	*calculate*	initial value, datatype
`STORAGE`	*read*, *store*	address, non-volatility, register flag, initial value
`EVENT`	*fire*, *count*, *interval*	—
`CHANNEL`	*transmit*, *receive*	carried datatypes
`STATE`	*set_state*; *in_state* predicate	—
`VALUE`	identity comparison	datatype
`ABSTRACT`	existence, identity comparison	—

2.2 Entity

Definition 2 (Entity)**.** *An entity is a tuple*

$$e = (\mathit{name},\ \mathit{type},\ \mathit{modifiers},\ \mathit{description})$$

where type $\in D$ *and modifiers is a set of key-value pairs drawn from the modifier schema of type.*

We write $\mathbf{E}$ for the finite set of all declared entities.

2.3 Extending the dictionary

A new entity type is added by declaring (i) what it *is*, (ii) what operations apply and their effect predicates, (iii) what modifiers are admitted. The rest of the formalism operates uniformly over whatever entries D contains.

3 Valuations and Traces

Definition 3 (Valuation)**.** *A valuation is a function* $\sigma : \mathbf{E} \to \mathbf{V}$ *assigning a value to every entity. For entities of type* `EVENT`*,* $\sigma(e) \in \{\mathit{true}, \mathit{false}\}$*, where* $\sigma(e) = \mathit{true}$ *iff the event occurs at that time point.*

We write Σ for the set of all valuations.

Definition 4 (Trace)**.** *A trace is a function* $\rho : \mathbf{T} \to \Sigma$*. We write* $\rho(t)(e)$ *for the value of entity* e *at time* t*.*

4 Expressions and Predicates

Definition 5 (Expression)**.** *The set* **Expr** *is the smallest set such that:*

- $\mathit{val}(e)$ *— current value:* $\rho(t)(e)$
- $\mathit{val}(e, t')$ *— value at absolute time* $t' \in \mathbf{T}$*:* $\rho(t')(e)$

- *val*$(e,\ @ev)$ — *value at last occurrence of* `EVENT` *ev:* $\rho(t_{last})(e),\ t_{last} = \max\{s \le t \mid \rho(s)(ev) = true\}$
- *prev*(e) — $\rho(t-1)(e)$ *(metric time only)*
- *at*$(e,\ n)$ — $\rho(t-n)(e),\ n \ge 1$ *(metric time only)*
- *addr*(e) — *static lookup of the address modifier on* e
- *diff*$(a,\ b)$, *signed*(a) *for* $a, b \in \mathbf{Expr}$
- *max_peak*(e), *min_peak*(e) — $\sup$ / $\inf\{\rho(s)(e) \mid s \le t\}$
- *count*(e) — *total occurrences of* `EVENT` e *up to* t
- *count*$(e,$ *since* $e')$, *count*$(e, [a, b])$ — *windowed counts*
- *interval*$(e,\ n)$ — *span between the 1st and* n*-th most recent occurrence of* `EVENT` e
- *any constant* $c \in \mathbf{V}$
- *arithmetic combinators* $+, -, \times, /$ *closed under* $\mathbf{V}$

Every $\varphi \in \mathbf{Expr}$ *has a denotation* $[\![\varphi]\!](\rho, t) \in \mathbf{V}$.

Definition 6 (Predicate)**.** *The set* **Pred** *is the smallest set such that:*

- *cmp*$(a,\ op,\ b)$ *for* $a, b \in \mathbf{Expr},\ op \in \{=, \neq, <, \le, >, \ge\}$
- *is*$(a,\ lit)$ *for* $a \in \mathbf{Expr}$, $lit \in \mathbf{V}$ — *value equals a named constant*
- *occurred*(e) *for* `EVENT` e — e *fires at the current time*
- *in_state*(s) *for* `STATE` s
- *all_of*(C), *any_of*(C) *for finite* $C \subseteq \mathbf{Pred}$
- *not*(ψ) *for* $\psi \in \mathbf{Pred}$
- $\forall x \in S : \psi(x),\ \exists x \in S : \psi(x)$ *for a finite set* S
- *boolean constants* $\top,\ \bot$

Every $\psi \in \mathbf{Pred}$ *has a denotation* $[\![\psi]\!](\rho, t) \in \{true, false\}$.

5 Actions

Definition 7 (Action)**.** *An action is a tuple* $\alpha = (op, args, pred)$ *where* op *is an operation drawn from the dictionary entry of the target entity,* $args$ *are its parameters, and* $pred \in \mathbf{Pred}$ *is the effect predicate.*

Operation	**Parameters**	**Effect predicate**
$calculate(e, expr)$	entity e, expression $expr$	$val(e) = [\![expr]\!]$
$read(e, a)$	entity e, address a	$val(e) = value\_at(a)$
$store(e, dst)$	entity e, destination dst	$value\_at(dst) = val(e)$
$invoke(e)$	entity e	$invoked(e)$
$hold(e)$	entity e	$halted(e)$
$toggle(e)$	entity e	$val(e) = \neg\, prev(e)$
$transmit(v, b, ch)$	`VALUE` v, binding b, `CHANNEL` ch	$transmitted(v, b, ch)$
$set\_state(s)$	`STATE` s	$in\_state(s)$

A *conditional action* $if(c,\, \alpha)$ has effect predicate $c \Rightarrow pred(\alpha)$.

6 Temporal Constructs

Temporal constructs compose predicates and actions into *trace predicates* — functions $P : \mathbf{Trace} \to \{true, false\}$. A trace ρ *satisfies* a requirement R (written $\rho \models R$) iff $P(\rho) = true$.

Construct	**ρ satisfies it iff**
$always(\psi)$	$\forall t \in \mathbf{T} : [\![\psi]\!](\rho, t)$
$eventually(\psi)$	$\exists t \in \mathbf{T} : [\![\psi]\!](\rho, t)$
$initial(\psi)$	$[\![\psi]\!](\rho, 0)$
$triggered(\psi_c, \psi_e)$	$\forall t : [\![\psi_c]\!](\rho, t) \Rightarrow \exists t' \geq t : [\![\psi_e]\!](\rho, t')$
$triggered\_within(\psi_c, \psi_e, d)$	$\forall t : [\![\psi_c]\!](\rho, t) \Rightarrow \exists t' \in [t,\, t{+}d] : [\![\psi_e]\!](\rho, t')$
$conjunction(P_1, P_2)$	
$disjunction(P_1, P_2)$	standard boolean composition
$negation(P)$	

6.1 Effect structures

When a requirement prescribes several actions, their relative timing is expressed as an *effect structure*.

Definition 8 (Effect structure)**.** *An effect structure is a pair* $S = (steps,\, constraints)$ *where* $steps = \{s_1, \ldots, s_n\}$ *is a finite set of actions and constraints is a set of temporal relations:*

Relation	***Meaning***
$precedes(s_i, s_j)$	s_i *completes before* s_j *begins*
$within(s_i, s_j, d)$	s_j *occurs within duration* d *after* s_i
$concurrent(s_i, s_j)$	*no ordering imposed*
$excludes(s_i, s_j)$	s_i *and* s_j *must not co-occur*
$immediately(s_i, s_j)$	s_j *at the next tick after* s_i*, with no intervening steps*
$conditional(c, s_i)$	s_i *required only when* c *holds*

Satisfaction. $\rho \models S$ iff there exists a mapping $\mu : steps \to \mathbf{T}$ such that $[\![pred(s_i)]\!](\rho, \mu(s_i))$ holds for each s_i and every relation in *constraints* is respected by μ.

7 Requirements

Definition 9 (Requirement)**.** *A requirement is a tuple* $R = (id, label, entities, constraint)$ *where:*

- *id is a unique requirement identifier*
- *label is a human-readable description*
- $entities \subseteq \mathbf{E}$ *— the entities participating in this requirement*
- *constraint is a trace predicate built from §§4–6*

A trace ρ satisfies R (written $\rho \models R$) iff $constraint(\rho) = true$. Entities declared in one requirement may be referenced by others; declaration conflicts are a static error.